The scan method takes a string or a regular expression argument and returns an array of all the times the string matches the regular expression. If the regular expression does not have groups, it's an array of the entire matched string. If the regular expression does have groups, it's an array of arrays containing the matches. For example:

```
"The pickaxe book".scan(/w+/)         # => []
"The pickaxe book".scan(/\s.{1}/)     # => [" p", " b"]
"The pickaxe book".scan(/(\s)(.{1})/) # => [[" ", "p"], [" ", "b"]]
```

Most generically, the split method allows you to split a string on an arbitrary value. When called with no arguments, it splits based on spaces. But you can call it with any string or regular expression argument. If the last substrings are empty, they are not part of the output:

```
"The pickaxe book".split       # => ["The", "pickaxe", "book"]
"The pickaxe book".split("e")  # => ["Th", " pickax", " book"]
"The pickaxe book".split(/\b/) # => ["The", " ", "pickaxe", " ", "book"]
"The pickaxe book".split("k")  # => ["The pic", "axe boo"]
```

An optional second argument is a limit. If the limit is greater than zero, the number of elements in the resulting array is limited to that number and the rest of the string comes together at the end. If the limit is negative, it has no effect, except that trailing empty substrings are part of the return value:

```
"The pickaxe book".split("e", 2)  # => ["Th", " pickaxe book"]
"The pickaxe book".split("k", -1) # => ["The pic", "axe boo", ""]
```

The split method can take an optional block, in which case the block is called once with each substring as an argument and the return values of the block calls are returned.

A special case of splitting is where you want to split the string exactly once. The partition method takes a string or regular expression as an argument and returns an array with three elements: the part of the string before the match, the match, and the remainder of the string. If the argument doesn't appear in the string, then the result is [string, "", ""]. The method rpartition does the same thing, but it finds the last match in the string, rather than the first:

```
"The pickaxe book".partition("e")  # => ["Th", "e", " pickaxe book"]
"The pickaxe book".rpartition("e") # => ["The pickax", "e", " book"]
```

Replacing Text in Strings

Ruby strings are mutable, meaning that the value of the string can change over time.

Partial String Assignment

The most general way to replace an arbitrary part of the string is by using square bracket syntax, []=. If a string being accessed with square brackets on the left side of an assignment statement, the right side of the statement replaces the referenced part of the string, even if the new part of the string is a different length than the substring. The return value of the assignment is the right side of the assignment, but the string itself changes:

```
x = "the pickaxe book"
x[12..] = "podcast"
x         # => "the pickaxe podcast"
```

This assignment works no matter which form you use inside the square brackets.